

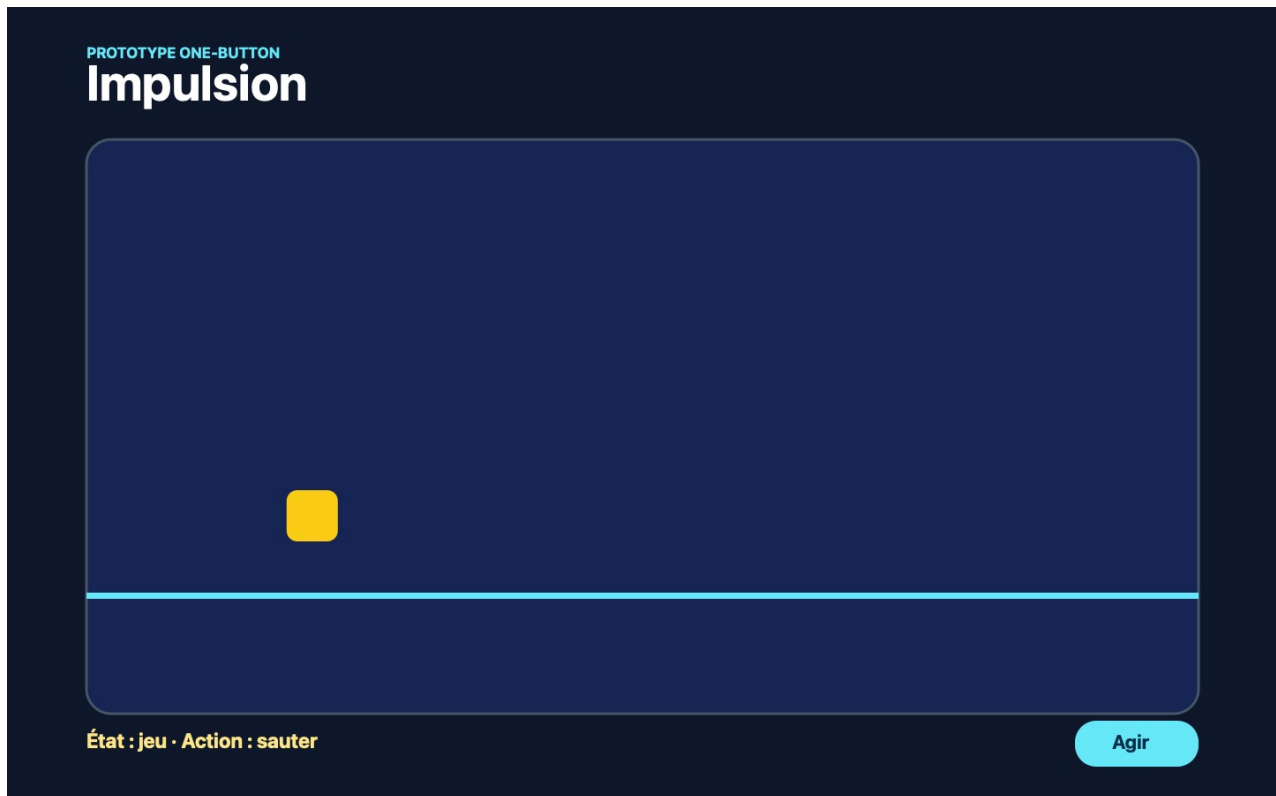
Atelier — One-button : une entrée, une décision

À ouvrir après 1-COURS.pdf du dossier projets/01-one-button/. 1–2 périodes. Canvas, Phaser et l'IA ne sont pas nécessaires dans cet atelier.

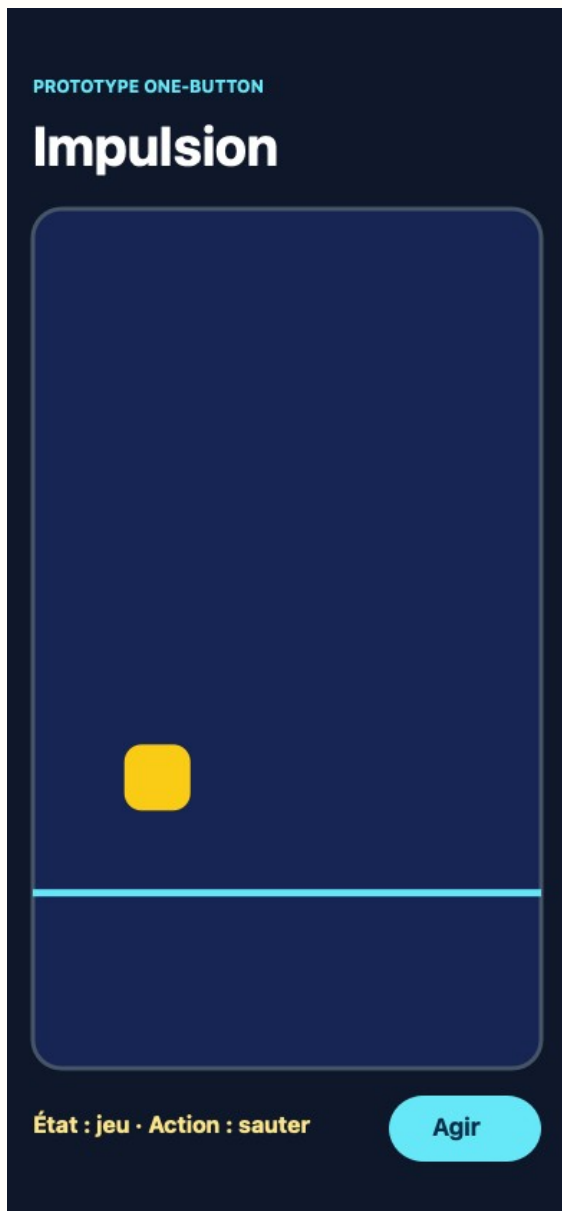
L'idée

Un jeu devient compréhensible quand la même entrée déclenche une action précise selon l'état courant. Ici, un seul bouton ouvre d'abord le jeu, puis fait sauter le joueur. Les couleurs et le nom du jeu peuvent changer : la règle reste **l'état décide du métier du bouton**.

À la fin, votre écran ressemble à ceci. Le rectangle jaune représente le joueur ; le texte sous la scène permet de vérifier l'état sans deviner.



Sur un écran de 390 pixels, la scène se resserre et le bouton reste visible.



Vous créez exactement ceci

```
m13/  
├─ index.html  
├─ concept.md  
├─ css/  
│ └─ style.css  
├─ images/  
├─ js/  
└─ jeu.js
```

`images/` reste vide pour ce premier prototype. Il existe déjà afin que les futurs visuels ne finissent pas à côté de `index.html`.

Étape 1 — Nommer la règle avant de coder

Vous avez déjà cliqué sur un bouton qui affiche d'abord une page, puis agit dans cette page. Ce n'est pas le bouton qui a changé : c'est l'état du site. Dans `concept.md`, écrivez seulement ces trois lignes :

But : déclencher un saut.
 Défaite : aucune dans ce prototype.
 Règle `actionDuBouton` : menu → jouer ; jeu → sauter.

Relisez la troisième ligne sans le nom de votre jeu. Si elle reste vraie, vous avez bien nommé la règle et non le décor.

Étape 2 — Poser une scène déjà visible

Créez l'arbre, puis placez dans `index.html` une page minimale avec ces quatre éléments dans `main` :

```
<h1>Impulsion</h1>
<div id="scene" aria-label="Zone de jeu">
  <div id="joueur"></div>
</div>
<p id="etat" aria-live="polite">État : menu</p>
<button type="button" id="action">Jouer</button>
```

Chargez `css/style.css` dans `head` et `js/jeu.js` juste avant `</body>`. Le CSS sert seulement à rendre la scène visible : un fond sombre, une bordure, un rectangle jaune pour `#joueur`, puis un bouton lisible. À ce stade, le clic ne fait encore rien ; c'est le bon résultat.

- [] Le titre, la scène, État : menu et le bouton Jouer sont visibles.
- [] La console du navigateur ne montre pas d'erreur de fichier manquant.

Étape 3 — Un premier cas seul : menu devient jeu

Ouvrez `js/jeu.js`. Un **état** est une donnée qui dit quelle scène est active. Commencez avec un seul cas afin de voir la transition avant d'ajouter le saut.

```
const etat = { scene: "menu" };
const bouton = document.getElementById("action");
const etiquette = document.getElementById("etat");

bouton.addEventListener("click", () => {
  if (etat.scene === "menu") {
    /* à vous : remplacer "menu" par "jeu" */
  }
  etiquette.textContent = `État : ${etat.scene}`;
});
```

Rechargez, puis cliquez une fois. Vous devez lire État : jeu. Si le texte reste sur menu, affichez `etat.scene` dans la console : vous vérifiez la donnée qui porte la décision.

Étape 4 — La règle `actionDuBouton`

Le premier clic sait maintenant ouvrir le jeu. Le clic suivant doit avoir un autre métier, sans ajouter un deuxième bouton. Créez la fonction nommée `actionDuBouton()` et complétez uniquement les deux

gestes laissés à votre charge.

```
function actionDuBouton() {
  if (etat.scene === "menu") {
    /* à vous : passer à l'état "jeu" */
    bouton.textContent = "Agir";
    return;
  }

  /* à vous : ajouter la classe "saute" à #joueur */
}
```

Remplacez le contenu du gestionnaire de clic par l'appel à `actionDuBouton()`, puis gardez la mise à jour de l'étiquette juste après. Dans `css/style.css`, donnez une position plus haute à `#joueur.saute`. Le premier clic affiche le jeu ; le deuxième déplace le rectangle jaune.

Étape 5 — Un geste bref, puis retour au repos

Une action qui reste bloquée en hauteur n'est plus un saut. Après l'ajout de la classe, utilisez `setTimeout` pour la retirer après environ 180 ms. La règle nommée **impulsion brève** devient : ajouter `saute`, attendre, retirer `saute`.

Testez au clic, puis avec Tab et Entrée. Un élément `button` reçoit déjà ces trois entrées sans fabriquer trois systèmes différents.

Réussi si

- [] Vous pouvez dire la règle `actionDuBouton` sans relire le fichier.
- [] Le premier clic fait passer menu à jeu.
- [] Les clics suivants déclenchent une impulsion brève.
- [] L'état affiché correspond à la donnée `etat.scene`.
- [] Le bouton fonctionne au clic, au toucher et avec Entrée.
- [] L'arbre contient `css/`, `images/` et `js/` aux chemins annoncés.
- [] La page reste lisible à 1440 et à 390 pixels.

AVEC L'IA

Montrez seulement `actionDuBouton()` à une IA et demandez : « Expliquez pourquoi le même bouton fait deux choses sans tester son texte. » Comparez sa réponse avec `etat.scene`. Ne lui demandez pas de refaire le prototype.

POUR LES RAPIDES

Ajoutez l'état `over`. Dans cet état, le même bouton affiche `Rejouer` et remet la scène sur `jeu`. Écrivez

d'abord la troisième branche dans `concept.md`.

Livrable

Rendez le dossier `m13/`. La page doit s'ouvrir par double-clic. À l'oral, montrez la donnée `etat.scene`, puis expliquez la règle `actionDuBouton`.